

tm:// Protocol Spec — Read-Only URI Namespace over MCP

Oct 6, 2026 · @Spice Y. Meatball

Purpose and scope

`tm://` exposes one server's websites, source repositories and filesystem as a single read-only URI namespace that Claude navigates over MCP. The point is to see both faces of a thing at once: the rendered page a visitor sees and the code, data and history that produced it.

Design decisions fixed for v1:

- **Read-only.** The server offers no verb that mutates anything. Safety lives in the verb set, not in a curated view.
- **The OS is the namespace.** The server exposes the real filesystem, repos and served sites, not a hand-built virtual tree. Structure is whatever is actually there.
- **Rules, not lists.** URI templates publish the grammar; the client constructs addresses it was never told about and tries them.
- **Negotiation is iterative.** Each `list` or `complete` answer hands back resolvable URIs for the next round. Coarse to fine, structure before content.
- **Standards first.** Where MCP, RFC 3986/6570/8288 or Plan 9 already name a pattern, use it rather than invent one.

Out of scope for v1: writes, command execution, proposals that materialize new routes (v2, see Open questions), and ROM-level access control (the owner stated this is not a concern for this design).

Concept to canonical pattern

Every idea from the design conversation maps to an existing, named pattern. Nothing in v1 is novel protocol; the novelty is the link resolver (see Mount contract).

Idea as stated	Canonical pattern	Where it is specified
Everything has a URI	MCP Resources: URI-addressed, read-only, each with a <code>contentType</code>	MCP spec, server features: resources
A handshake that says what we have	MCP <code>initialize</code> + capability negotiation; server <code>capabilities</code> field carries orientation (the MOTD)	MCP spec, lifecycle
Generate a URI from rules	MCP Resource Templates (<code>resources/templates/list</code>) using RFC 6570 URI templates	MCP spec; RFC 6570
An OS-level <code>ls</code> , the safe stuff	<code>resources/list</code> with opaque cursor pagination	MCP spec, pagination
That resolves into the pattern	<code>resources/read</code> returning text or blob + <code>contentType</code>	MCP spec
A negotiation, more than one round	<code>completion/complete</code> on template arguments, fed from live data	MCP spec, completions
This is what I would like to have (read side)	Templates + completions; a URI the client composes and tries	MCP spec
The OS is the namespace	Plan 9 / 9P: one tree, heterogeneous backends mounted into it; MCP Roots for client-declared scope	Plan 9 papers; MCP roots
Page to source to history joins	HATEOAS; typed <code>rel</code> links (Web Linking)	RFC 8288
Facets like <code>.source</code> , <code>.log</code> , <code>.raw</code>	Representation negotiation by <code>contentType</code> , or distinct mounts, never path suffixes	RFC 3986; HTTP content negotiation
Token in the URL	Bearer token in a header; OAuth 2.1 when a remote client needs it	MCP spec, authorization; RFC 6750
Later: make the thing I asked for	MCP Tools + Elicitation (v2)	MCP spec

Verify exact method names against the current revision at modelcontextprotocol.io before coding; the primitives above have been stable but field names drift between revisions.

Layering

Three read-only verbs sit on a published grammar over one namespace

6 • Links

Typed rel links on every read: generated-by, source, history, renders-as (RFC 8288)

5 • Templates

RFC 6570 grammar published by the server; completions fill arguments from live data

4 • Verbs

list · read · complete — nothing else. Read-only by construction, not by policy

3 • Namespace

tm://host/MOUNT/... — mounts: fs, git, http; listed at the root, added without protocol change

2 • Session

MCP initialize, capabilities (resources, completions), instructions = orientation

1 • Transport

Streamable HTTP with bearer auth; reachable from claude.ai and desktop, not only local stdio

protocol layering · 6 layers, verbs highlighted

Each layer depends only on the one below it. The verb layer is the safety boundary: with no mutating verb defined, no higher layer can mutate.

URI grammar

One scheme, `tm`, and the first path segment names the mount. Everything after the mount is that backend's native path, unchanged. RFC 3986 syntax throughout; percent-encode anything a backend path contains that the URI grammar reserves.

```
tm://<host>/<mount>/<backend-path>[?<facet-params>]
```

Part	Rule	Example
host	The server instance. One server may serve several hosts; <code>list</code> at <code>tm://</code> enumerates them.	box
mount	A backend type plus optional instance: <code>fs</code> , <code>git/<repo></code> , <code>http/<site></code> . Listed at <code>tm://<host>/</code> .	<code>git/autopsy</code> , <code>http/tinymachines.ai</code>
backend-path	Whatever the backend calls it: a filesystem path, a git tree path, an HTTP path. Trailing <code>/</code> means a directory or collection; <code>list</code> works on it.	<code>/srv/autopsy/src/patterns.rs</code> , <code>/autopsy/patterns</code>
facet-params	Representation, not identity. Which bytes of the same thing: raw, rendered, log, blame, at a ref.	<code>?at=main</code> , <code>?as=log&n=20</code>

Representation stays in the query string so that one thing keeps one identity.

`tm://box/git/autopsy/src/patterns.rs` is the file; `?at=v0.3` is the same file at a tag; `?as=blame` is a different representation of it. Path suffixes like `.log` would mint a second identity for the same thing and break the joins.

Templates the server publishes (RFC 6570):

```
tm://{host}/
tm://{host}/fs{/path*}
tm://{host}/git/{repo}/{/path*}{?at,as,n}
tm://{host}/http/{site}/{/path*}{?as}
```

Completions are wired per argument: `repo` from the repos the server can see, `path` from a directory listing under the prefix typed so far, `at` from branches and tags, `site` from the vhosts the web server serves.

Defined `as` values: `raw` (default; bytes with the backend's mimeType), `rendered` (http only; the HTML a visitor gets), `text` (http only; readable extraction of the rendered page), `log`, `blame`, `tree` (git only), `stat` (fs/git; metadata as JSON, no content).

Resolution failures return a structured error, never a bare 404: `not-found` carries up to five nearest sibling URIs so a wrong guess teaches the client instead of dead-ending; `too-large` carries the byte size and a `?range=` template; `binary` carries the mimeType and a `?as=stat` link.

Mount contract

A mount is any backend that implements three operations and one optional one. Adding a backend is adding a mount; the protocol, the templates and the client do not change.

Operation	Input	Output	Required
<code>list(path, cursor)</code>	A collection path	Children as URIs with <code>name</code> , <code>mimeType</code> , <code>size</code> , <code>isCollection</code> ; a <code>nextCursor</code> when truncated	yes
<code>read(path, facet)</code>	A leaf path + facet params	Content (text or base64 blob) + <code>mimeType</code> + a <code>links</code> array (see Link vocabulary)	yes
<code>complete(arg, prefix)</code>	A template argument name + what the client typed	Up to 100 candidate values, <code>hasMore</code> flag	yes
<code>links(path)</code>	Any path	The same <code>links</code> array <code>read</code> would attach, without the content	optional; lets a client pivot without downloading

Mounts for v1:

- **fs** — the host filesystem under one or more configured roots. `list` is a directory read. `read` returns file bytes with a sniffed mimeType. `complete` on `path` is a directory listing filtered by prefix. Links: `history` when the file sits inside a git working tree (points into the matching `git` mount), `renders-as` when a build manifest says the file produced a served page.

- **git** — one mount per repository. Paths are tree paths at a ref; `at` defaults to the checked-out HEAD. `read` with `as=log` returns commits touching the path, `as=blame` returns blame, `as=tree` returns the tree listing as JSON. Links: `working-copy` (the fs URI if the repo is checked out), `renders-as`, `generated-by`.
- **http** — one mount per served site. Paths are URL paths. `read` fetches from the local web server (loopback, not the public internet) and returns `rendered` HTML or a `text` extraction. Links: `source` (the template or route handler), `generated-by` (the analysis output a page was built from), `data` (the JSON/DB the page reads).

The link resolver

The only part of this design with no off-the-shelf answer is how the `http` mount knows that `/autopsy/patterns` was produced by `src/autopsy/patterns.rs` and reads from `data/patterns.duckdb`. Three sources, tried in order:

1. **Build manifest** (preferred). The site build emits `tm-links.json`: one entry per route with its `source`, `generated-by` and `data` paths. Exact and cheap; costs one build step.
2. **Route table introspection**. The server asks the running app for its route table and maps handlers to source files by symbol. Works for frameworks that expose it; approximate for template-rendered pages.
3. **Heuristic**. Path-shape matching (`/autopsy/patterns` to `**/autopsy/patterns.*`). Always labelled `confidence: low` in the link.

Every link carries `confidence` (`exact`, `inferred`, `low`) so the client knows how far to trust a pivot.

Common requirements on every mount

- Paths are canonicalized before any filesystem call; `..` and symlinks that resolve outside a mount root are refused with `out-of-root`.
- `read` enforces a byte cap (default 256 KiB); larger content returns `too-large` with a `?range=` template.
- Binary content is returned as a blob only when the client asked for `as=raw` on a non-text mimeType; otherwise `binary` with a `stat` link.
- Every `list` paginates with an opaque cursor; page size is server-chosen, at most 500.

Link vocabulary

Links are how the client pivots between faces of a thing without guessing paths. Every `read` response carries a `links` array; each link is `{rel, href, confidence, title?}` following RFC 8288 semantics. Where IANA already registers a relation, reuse it; the rest are namespaced `tm:` relations.

rel	From	To	Meaning
tm:source	http page	git or fs file	The template, route handler or generator that emitted this page
tm:renders-as	git or fs file	http page	Inverse of tm:source
tm:generated-by	http page, data file	git file or analysis output	The computation whose output this is (an analysis pass, a build step)
tm:generates	git file	http page, data file	Inverse of tm:generated-by
tm:data	http page	fs or git data file	The dataset the page reads at render time
tm:working-copy	git path	fs path	The checked-out file for this tree path
tm:repository	fs path	git path	Inverse of tm:working-copy
version-history (IANA)	any	git path ? as=log	Commits touching this thing
latest-version (IANA)	git path ? at=<ref>	git path at HEAD	Newest revision
alternate (IANA)	any	same path, other as=	Another representation of the same identity
up (IANA)	any	parent collection	One level up
collection (IANA)	any leaf	the listing it belongs to	Its siblings
describedby (IANA)	any	?as=stat	Metadata without content

Rules:

- Links are typed, never free-text. A client that does not know a rel ignores it.
- Inverse pairs are both emitted, so a traversal can be walked in either direction.

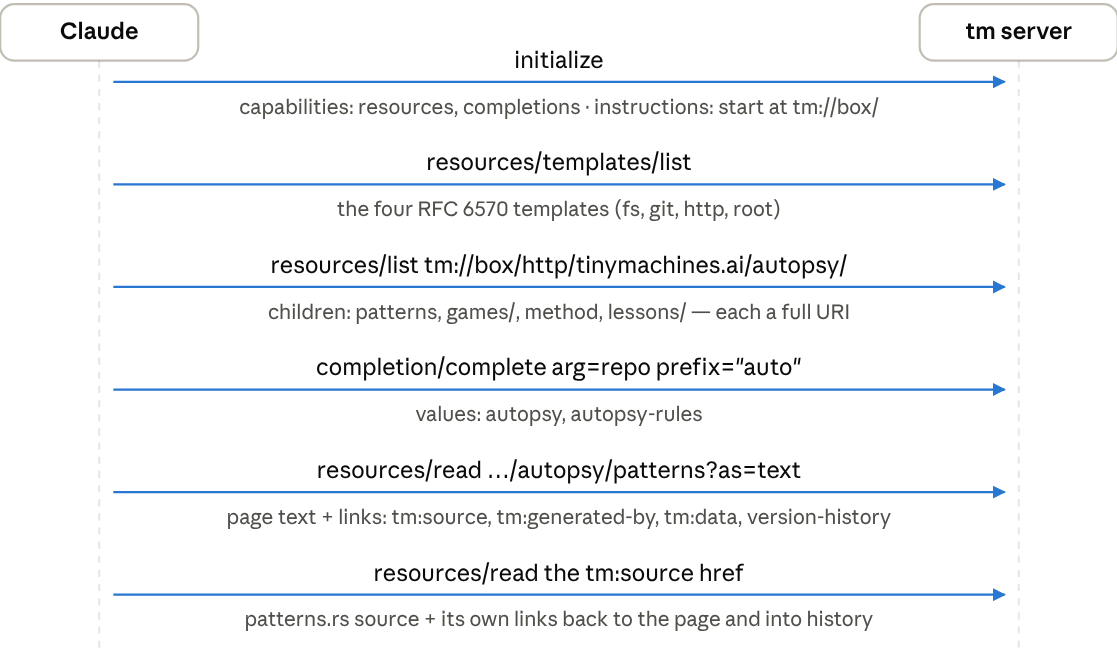
- `confidence` is mandatory on `tm:` relations (`exact` , `inferred` , `low`) and omitted on IANA relations, which are always exact.
- A link's `href` is a complete `tm://` URI, never a relative path; relative resolution across mounts is undefined.

Example: reading `tm://box/http/tinymachines.ai/autopsy/patterns?as=text` returns the page text plus

```
tm:source      tm://box/git/tinymachines-
site/src/routes/autopsy/patterns.rs  exact
tm:generated-by tm://box/git/autopsy/src/rules/mod.rs
exact
tm:data        tm://box/fs/srv/autopsy/out/patterns.parquet
exact
version-history tm://box/git/tinymachines-
site/src/routes/autopsy/patterns.rs?as=log
alternate      tm://box/http/tinymachines.ai/autopsy/patterns?as=rendered
up             tm://box/http/tinymachines.ai/autopsy/
```

Session walkthrough

Each answer hands back URIs for the next round



Six exchanges take the client from a cold connection to the source file behind a rendered page. No path was guessed: the root was named in `instructions`, the templates supplied the grammar, `list` and `complete` supplied real values, and the final pivot followed a typed link. Every response is navigable, so a seventh exchange can continue from any href already in hand.

Safety and auth

Read-only removes write risk; it does not remove disclosure or availability risk. These controls are the minimum for a server that walks a real filesystem.

Concern	Control
Escape from a mount root	Canonicalize (<code>realpath</code>) before every fs call; refuse <code>out-of-root</code> . Symlinks resolving outside the root are refused, not followed.
Unbounded reads	256 KiB default byte cap per <code>read ; ?range=</code> for larger files; <code>list</code> pages at most 500 entries.
Secrets in the tree	Deny-list of path patterns applied before any read (<code>.env</code> , <code>*.pem</code> , <code>id_*</code> , <code>*secret*</code> , <code>.git/config</code> credentials). Refused as <code>denied</code> , listed as <code>redacted</code> so the client knows they exist.
Token exposure	Bearer token in the <code>Authorization</code> header, never in the URI. Scope the token to <code>resources:read</code> and <code>completion</code> ; rotate on a schedule. Upgrade to OAuth 2.1 when a second client or user appears.
Loopback-only http mount	The <code>http</code> mount fetches from <code>127.0.0.1</code> or a Unix socket, not from public DNS, so the server cannot be used as an open proxy.
Request abuse	Per-token rate limit; <code>complete</code> capped at 100 values; <code>list</code> recursion is never offered (the client walks one level at a time).
Audit	Every <code>read</code> logged with URI, facet, bytes returned, token id. Log is itself readable at <code>tm://box/fs/var/log/tm/</code> so the client can see what it did.

The permission model is three allowed verbs and a deny-list, not a curated namespace. That keeps the surface small enough to reason about and means a new mount inherits every control above with no extra work.

Open questions and v2

Decisions still to make before coding:

- ❑ **Resources vs. a `resolve` tool shim.** MCP clients differ in how proactively they surface resources; some are tool-first. Test in `claude.ai` and Claude desktop early. If resources are not navigated fluidly, add one read-only tool, `resolve(uri)`, that wraps `read` and returns content plus links. It changes nothing about the namespace.
- ❑ **Manifest format for the link resolver.** Settle the `tm-links.json` schema and which build step emits it. This gates `confidence: exact` on every `tm:` link.
- ❑ **Multi-host.** One server per box, or one server fronting several boxes? The grammar allows both; the auth story is simpler per box.
- ❑ **Subscriptions.** `resources/subscribe` on a path would let the client notice a file changed mid-conversation. Cheap for fs (`inotify`), awkward for http. Defer unless a use appears.
- ❑ **Language.** Rust is the natural fit given the surrounding stack; the MCP Rust SDK should be checked for Streamable HTTP and completions support before committing.

v2, deliberately excluded from this spec:

- **Proposals.** The "this is what I would like to have" direction: a Tool that accepts a URI the client wishes existed plus a description, stages a scaffold (route, template, analysis stub) as a branch or diff, and returns a review URI. Nothing lands without the owner's approval; the Tool is proposal-shaped, modelled on a pull request.
- **Elicitation.** When a proposal is ambiguous, the server asks the client structured questions through MCP elicitation rather than guessing.
- **Write verbs.** None planned. Mutation stays behind proposals.

Sources: [MCP specification](#) · [RFC 3986 URI syntax](#) · [RFC 6570 URI Template](#) · [RFC 8288 Web Linking](#) · [RFC 6750 Bearer Token Usage](#) · [IANA link relations](#)